SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Simulation

Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (192511167)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

SIMATS ENGINEERING
ASSESSMENT TOOL

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non-determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b)

W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/proble M	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized Answer	Understandabl e with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 1: Non-Deterministic Finite Automaton (NFA) for an Automatic Door System

Aim: To design an NFA representing an automatic door using sensor inputs P (person detected) and N (no person detected).

States

- \(q_0\): Closed — initial state
- \(q_1\): Opening
- \(q_2\): Open
- \(q_3\): Closing

Initial state: \(q_0\)

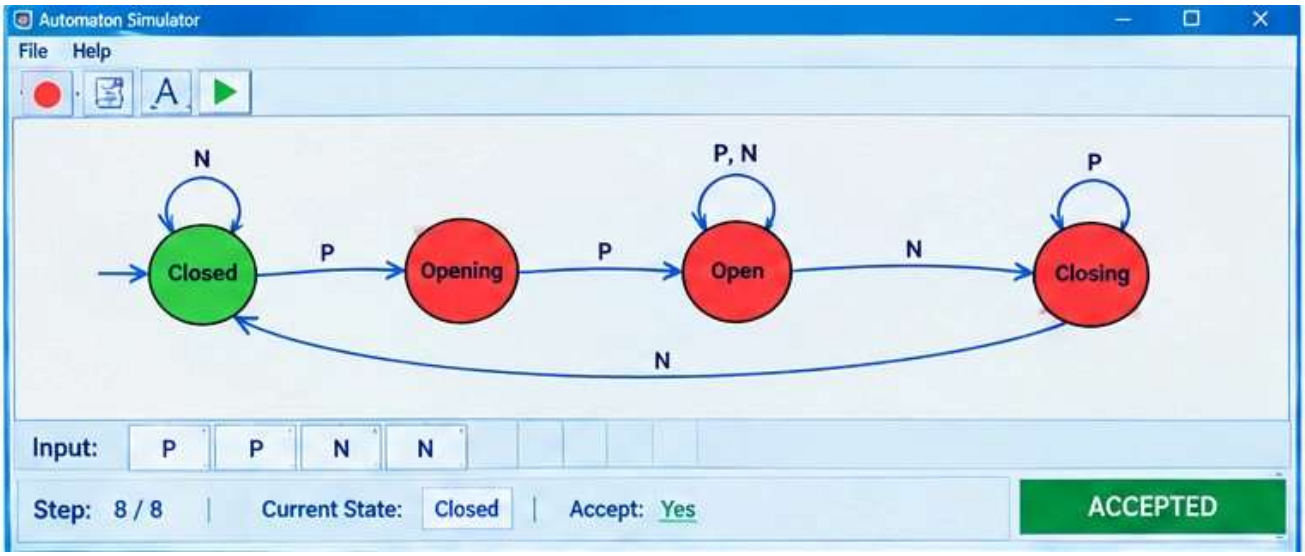
Final state: \(q_2\) (door is open)

Transition table

Current state	Input P	Input N
→ \(q_0\) Closed	\(q_1\)	\(q_0\)
\(q_1\) Opening	\(q_2\)	\(q_3\)
*\(q_2\) Open	\(q_2\) or \(q_3\)	\(q_3\)
\(q_3\) Closing	\(q_0\)	\(q_0\)

For an NFA, the Open state can have multiple possible next states for the same input. In particular, on P, the door may remain Open or begin Closing.

State Diagram



SIMATS ENGINEERING
ASSESSMENT TOOL

Sample input and output

Input	Expected door behavior
P	Closed → Opening
PP	Closed → Opening → Open
PPN	Closed → Opening → Open → Closing
PPNN	Door begins closing and returns to Closed

Result: The NFA models the door's movement using person-detection sensor inputs.

Question 2: Finite Automaton for a Traffic Light Control System

Aim: To design a finite automaton that cycles through Red, Green, and Yellow traffic-light states in the correct order.

States

- \(q_0\): Red
- \(q_1\): Green
- \(q_2\): Yellow

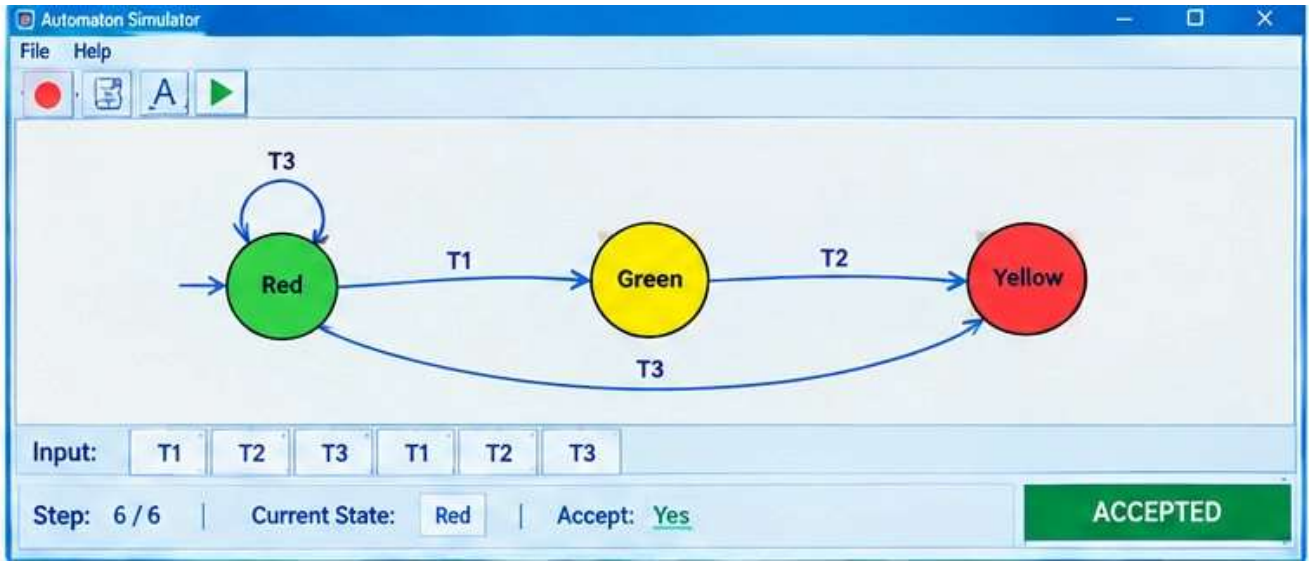
Input symbol: T (timer expires).

Initial state: \(q_0\)

Transition table

Current state	Input	Next state
→ \(q_0\) Red	T	\(q_1\) Green
\(q_1\) Green	T	\(q_2\) Yellow
\(q_2\) Yellow	T	\(q_0\) Red

State diagram



Timing constraints and extensions

- Red remains active for the configured red-light duration.
- Green remains active for the configured green-light duration.
- Yellow remains active for the configured yellow-light duration.
- When a timer expires, the automaton moves to the next state and continuously repeats the cycle.
- Pedestrian crossing: Add states for Walk and Don't Walk, coordinated with the vehicle lights.
- Emergency override: Add a priority control input that transitions to a safe traffic state, such as Red for conflicting traffic, subject to safe transition rules.

A basic finite automaton represents the sequence; a timed automaton or timer-controlled implementation is needed to enforce actual durations.

Result: The automaton models the continuous traffic-light sequence Red → Green → Yellow → Red.

Question 3: Design a DFA to Accept Strings Ending with ab

Aim: To construct a DFA over the alphabet \(\Sigma=\{a,b\}\) that accepts strings ending in ab.

Given input string: aaabab

States

- \(q_0\): Start state; the current string does not end in a or ab.
- \(q_1\): The latest symbol is a.
- \(q_2\): The string ends with ab — accepting state.

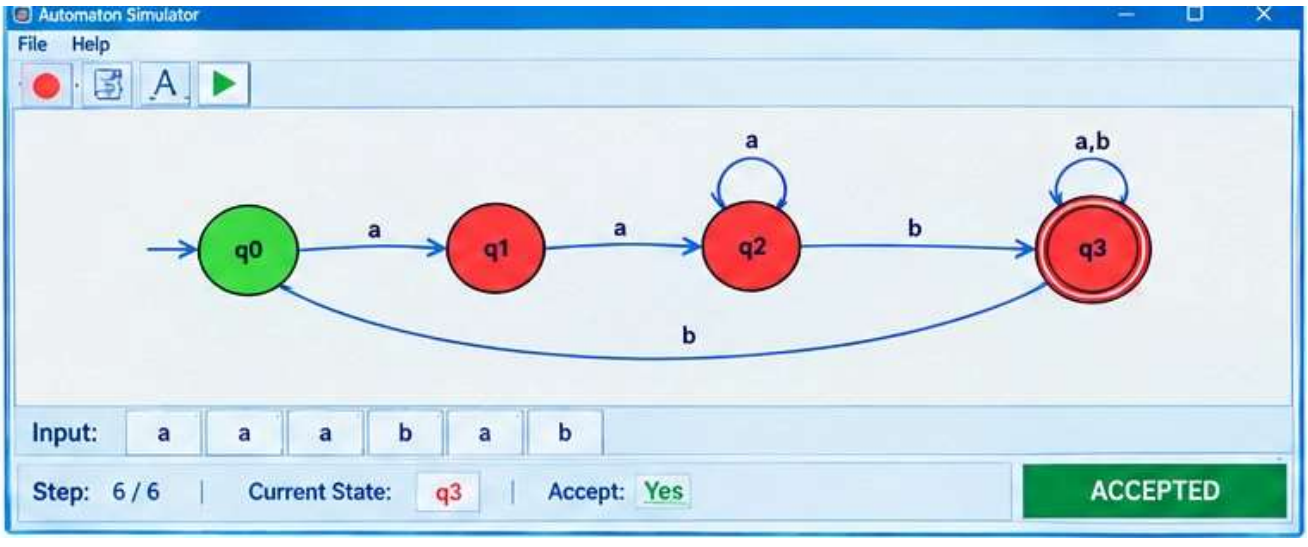
Initial state: \(q_0\)

Final state: \(q_2\)

Transition table

Current state	Input a	Input b
→ \(q_0\)	\(q_1\)	\(q_0\)
\(q_1\)	\(q_1\)	\(q_2\)
*\(q_2\)	\(q_1\)	\(q_0\)

State diagram



SIMATS ENGINEERING
ASSESSMENT TOOL

Simulation of aaabab

Step	Read symbol	State
0	—	\(q_0\)
1	A	\(q_1\)
2	A	\(q_1\)
3	A	\(q_1\)
4	B	\(q_2\)
5	A	\(q_1\)
6	B	\(q_2\)

Final state: \(q_2\), which is accepting.

Result: aaabab is ACCEPTED because its last two symbols are ab.

Question 4: DFA to Accept Only c, bc, and bcaaa

Aim: To design a DFA over the alphabet \(\Sigma=\{a,b,c\}\) that accepts exactly the three strings c, bc, and bcaaa.

States

- \(q_0\): Initial state
- \(q_1\): Read b
- \(q_2\): Read bc — accepting state
- \(q_3\): Read bca
- \(q_4\): Read bcaa
- \(q_5\): Read bcaaa — accepting state
- \(q_6\): Dead state
- \(q_7\): Read c directly — accepting state

Initial state: \(q_0\)

Final states: \(F=\{q_2,q_5,q_7\}\)

Transition table

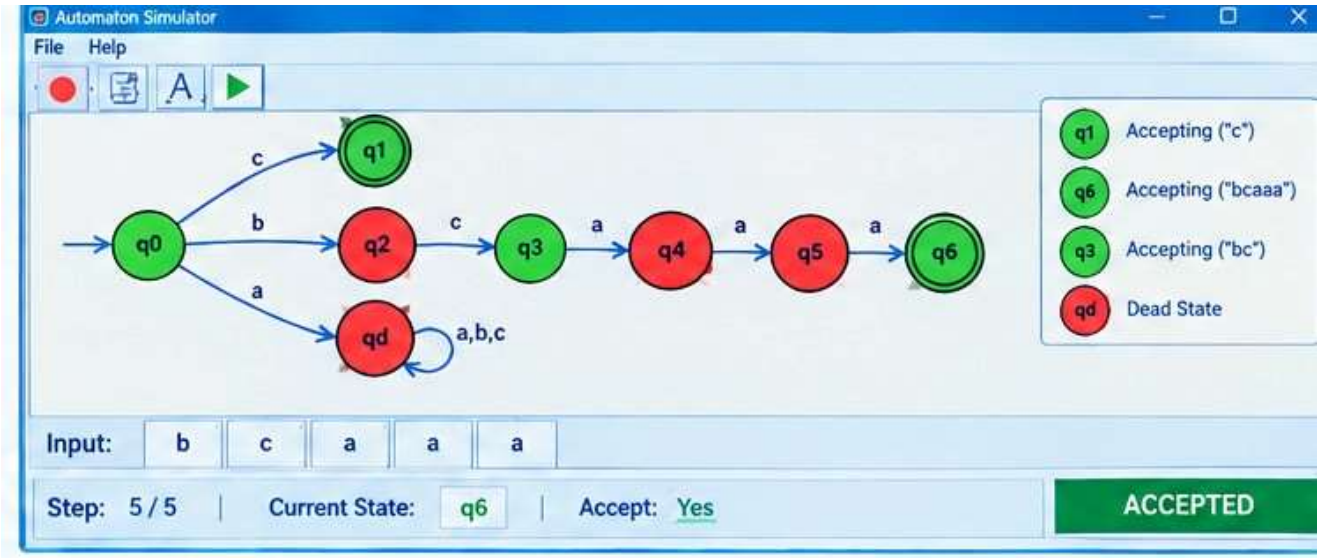
State	a	B	C
→ \(q_0\)	\(q_6\)	\(q_1\)	\(q_7\)
\(q_1\)	\(q_6\)	\(q_6\)	\(q_2\)
*\(q_2\)	\(q_3\)	\(q_6\)	\(q_6\)
\(q_3\)	\(q_4\)	\(q_6\)	\(q_6\)
\(q_4\)	\(q_5\)	\(q_6\)	\(q_6\)
*\(q_5\)	\(q_6\)	\(q_6\)	\(q_6\)
\(q_6\)	\(q_6\)	\(q_6\)	\(q_6\)
*\(q_7\)	\(q_6\)	\(q_6\)	\(q_6\)

SIMATS ENGINEERING
ASSESSMENT TOOL

* denotes an accepting state.

State diagram

Follow the main path for bcaaa; the direct branch accepts c.



Test the inputs

Input string	Output
c	ACCEPTED
bc	ACCEPTED
bcaaa	ACCEPTED
bca	REJECTED
bcaab	REJECTED
a	REJECTED
bb	REJECTED

Result: The DFA accepts only the three specified strings and rejects all other strings over the given alphabet.

Question 5: DFA for All Non-Empty Strings Beginning with a or b

Aim: To design a DFA over $\Sigma=\{a,b\}$ that accepts every non-empty string starting with either a or b.

States

- q_0: Initial state; no input has been read.
- q_1: Accepting state; at least one valid symbol has been read.
- q_d: Dead state for invalid symbols, if the simulator supports them.

Initial state: q_0

Final state: $F=\{q_1\}$

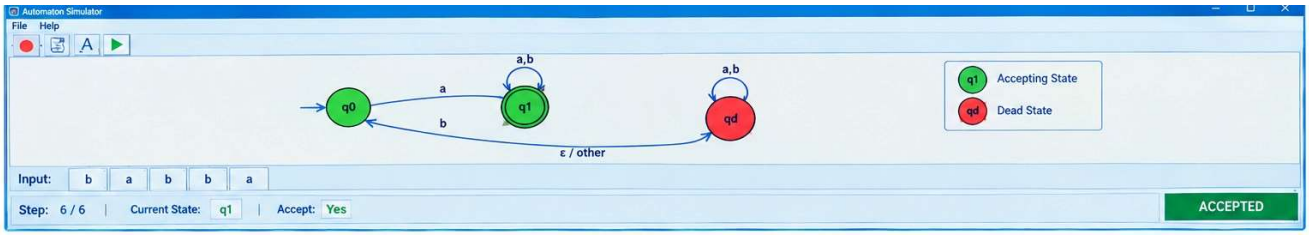
SIMATS ENGINEERING
ASSESSMENT TOOL

Transition table

Current state	Input a	Input b
→ \(q_0\)	\(q_1\)	\(q_1\)
*\(q_1\)	\(q_1\)	\(q_1\)
\(q_d\)	\(q_d\)	\(q_d\)

The dead state is optional for this particular alphabet because both a and b are valid symbols. If the simulator accepts inputs outside \(\{a,b\}\), those inputs must be rejected.

State diagram



Sample input and output

Input	Output
a	ACCEPTED
b	ACCEPTED
ab	ACCEPTED
baab	ACCEPTED
aaaa	ACCEPTED
bbbb	ACCEPTED
ε (empty string)	REJECTED

Result: The DFA accepts all non-empty strings over \(\{a,b\}\), because every such string necessarily begins with either a or b.